DURCL: Automated Distribution-Level Rebucketing Curriculum Learning for RL-Based LLM Post-Training

DSAA3053 Course Project

April 28, 2026

Abstract

Reinforcement-learning post-training for large language models often mixes heterogeneous environments whose difficulty and usefulness vary over training. Uniform sampling ignores this nonstationarity. This paper studies DURCL, Automated Distribution-Level Rebucketing Curriculum Learning, an online curriculum mechanism that clusters Math and Zebra prompts by difficulty, samples clusters with an Upper Confidence Bound (UCB)-style policy, and periodically refines clusters through conservative micro-bucket rebucketing. DURCL is implemented on top of a DUMP/verl GRPO training path. Current evidence supports a modest but useful conclusion: DURCL produces interpretable adaptive bucketing behavior and selective gains on the Math+Zebra setting. A no-rebucket scheduler improves step-200 validation accuracy over random sampling on both Math and Zebra, and its UCB scores drift from low-index to high-index clusters over training, showing that the sampler is active rather than fixed. Accuracy-initialized no-rebucket validation further improves to 0.500 on Math and 0.300 on Zebra at step 200, while the corresponding rebucketing result is still pending. These results suggest that learner-state-aware environment selection is promising, while stricter evaluation remains necessary.

1 Introduction

RL-based LLM post-training increasingly relies on heterogeneous mixtures of tasks, datasets, and difficulty levels. This project focuses on a two-source mixture: Math problems and Zebra logic puzzles. Sampling these environments uniformly is simple, but it assumes that every distribution is equally useful at every point in training. This assumption is unlikely to hold: some distributions may be already solved, some may be too difficult to provide useful gradient signal, and some may become useful only after the model reaches an intermediate competence level.

The issue is visible in GRPO-style training. Since advantages are computed relative to a group of sampled rollouts, both too-easy and too-hard prompts can produce little useful update: if all rollouts are correct or all rollouts fail, the within-group learning signal is small. The most useful data therefore changes as the model improves. DURCL treats this as a non-stationary bandit problem: each curriculum cluster is an arm whose usefulness changes over training.

This project follows the *Algorithm Development* track in the course guideline. The algorithmic contribution is DURCL, an adaptive environment-selection module for RL-based LLM post-training. The evaluation contribution is a numerical benchmark and diagnostic analysis comparing random sampling, DURCL sampling, and rebucketing variants on the checked-in Math+Zebra results.

Curriculum learning argues that example order and difficulty can affect optimization [2]. Recent work such as DUMP extends this view to RL-based LLM post-training by using advantage magnitude as a distribution-level learnability signal and UCB as an adaptive scheduling rule [6]. This project asks whether a related idea can be made practical in a Math+Zebra reasoning setting: can online advantage signals guide environment selection and correct imperfect static difficulty buckets?

We implement and evaluate DURCL. DURCL maintains ordered difficulty clusters, samples clusters using UCB with a probability floor, tracks decayed absolute advantage dynamics, and periodically performs low-frequency micro-bucket rebucketing. The design intentionally prioritizes interpretability and guardrails over aggressive dynamic reclustering.

The main contribution of this project is not a claim of uniform performance improvement. Instead, the contribution is a repo-backed analysis of where adaptive environment selection already works and where it fails. The strongest current evidence is structural: DURCL changes sampling and bucket composition in interpretable ways. The downstream performance evidence is still incomplete, which motivates accuracy-based rebucketing evaluation and stricter multi-seed testing.

2 Background

Curriculum learning. Curriculum learning studies whether presenting easier or more informative examples earlier can improve training dynamics [2]. For LLM post-training, the relevant curriculum is not only over scalar difficulty; it is also over task source, reward type, response format, and the model's evolving competence.

Bandit scheduling. UCB algorithms address the exploration-exploitation tradeoff in multi-armed bandits by preferring arms with high empirical value or high uncertainty [1]. DURCL treats each cluster as an arm in a non-stationary bandit and uses an advantage-derived cluster value plus a UCB bonus to determine sampling probabilities.

RL-based LLM post-training. The training scripts in this repository use verl with GRPO-style advantage estimation on Qwen2.5 model checkpoints [4, 3, 5]. The project uses the DUMP/verl training path as infrastructure, while the DURCL scheduler is implemented as a separate module.

3 Problem Setting

The task is RL-based post-training over a two-source Math+Zebra reasoning mixture. Math provides mathematical problem-solving prompts, while Zebra provides logic-puzzle prompts. The two sources have different difficulty structures and different initial model competence, making them a useful minimal setting for studying adaptive environment selection without introducing additional task families.

Table 1: Math+Zebra task setting used by the project.

Task	Difficulty labels	Target behavior
Math	d1–d5	mathematical problem solving
Zebra	d1–d4	logic-puzzle answer selection

The initial experiments use raw difficulty labels to form curriculum clusters. Zebra has four difficulty levels, while Math has five, so the highest-index cluster is Math-only at initialization. This asymmetry is important for interpreting later composition drift. The random baseline samples from the same Math+Zebra pool without curriculum scheduling, making it the direct comparison point for DURCL variants.

4 Method

DURCL consists of initial bucketing, UCB cluster scheduling, exponential-decay advantage tracking, and micro-bucket rebucketing. Figure 1 summarizes the feedback loop.

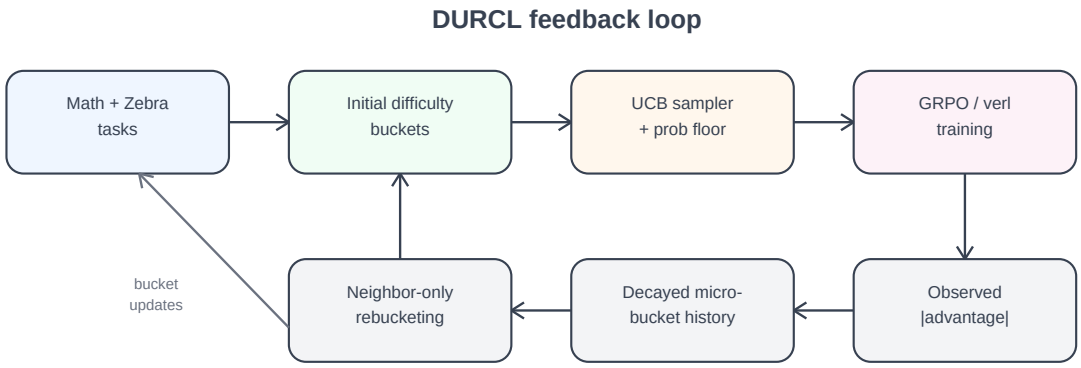

Figure 1: DURCL feedback loop. Math and Zebra examples are placed into initial difficulty buckets. A UCB sampler selects clusters for training. Observed absolute advantages update decayed micro-bucket histories, and persistent structural mismatches are corrected through neighbor-only rebucketing.

From DUMP to DURCL. DURCL starts from the DUMP idea that absolute advantage can measure distribution learnability [6]. In the original DUMP algorithm, each distribution d_j stores a sliding window $A^w_{d_j}$ of recent $|\hat{A}(o)|$ values, keeps the last k entries, and uses the resulting distribution value in UCB scheduling. This works naturally when each distribution is internally coherent.

The Math+Zebra setting introduces two additional difficulties. First, the scoring rules and output formats are different, so absolute advantage is not perfectly comparable across task sources. Second, problem types differ enough that the model may learn them at different rates. A cluster that is initially coherent under nominal difficulty labels can become internally mixed as Math and Zebra drift differently. DURCL therefore makes two changes. It replaces raw sliding windows with exponential-decay states for smoother and cheaper tracking, and it adds micro-bucket rebucketing to repair local structural mismatches inside clusters.

Initial clusters. The scheduler accepts explicit `cluster_id` or `difficulty_band` fields. It also supports a calibration map where higher measured accuracy maps to easier initial clusters. The current reported experiments mainly use coarse difficulty-based initialization, so bucket assignment is intentionally treated as imperfect.

Exponential-decay advantage state. For each sampled unit, DURCL maintains an exponential moving average of absolute advantage:

$$s_i \leftarrow \lambda s_i + (1-\lambda)|A_i|, \tag{1}$$

where λ is the decay parameter and A_i is the observed advantage. Compared with a fixed-size sliding window, the decayed state requires less retained history and makes the curriculum signal smoother. Cluster values are computed from recently active advantage states.

UCB sampling. For cluster c, DURCL computes

$$\mathrm{UCB}_c = v_c + \beta\sqrt{\frac{2\log(N+1)}{n_c+1}}, \tag{2}$$

where v_c is the active cluster value, N is total draws, and n_c is draws from cluster c. UCB scores are passed through a softmax and mixed with a probability floor so every nonempty cluster remains sampleable.

4.1 Micro-bucket Rebucketing

The cluster scheduler assumes that each curriculum cluster is internally coherent. However, this assumption can be violated when the initial clusters are built from coarse difficulty labels or noisy pre-evaluation signals. To address this, DURCL uses a conservative micro-bucket rebucketing mechanism that refines cluster membership online without turning the scheduler into an unconstrained clustering algorithm.

The motivation is to separate two roles. The UCB scheduler should decide which cluster is currently useful for training; it should not be responsible for repairing every bad initial assignment. Rebucketing is a slower structural correction layer. It only checks whether small groups inside a cluster have dynamics that consistently look more similar to an adjacent cluster.

For each initial task–cluster group, we randomly split its examples into several micro-buckets. A micro-bucket is the smallest unit allowed to move; individual samples are never moved independently. This reduces the variance of sample-level advantage estimates and makes rebucketing a structural correction rather than an instance-level filtering procedure.

During training, each micro-bucket records the mean absolute advantage of its sampled examples. Given a recent observation window, we summarize the learning dynamics of micro-bucket G_g by

$$z_g = (\ell_g, s_g), \tag{3}$$

where ℓ_g is the recent average absolute advantage and s_g is the fitted slope of the advantage trajectory. Thus, ℓ_g measures the current strength of the learning signal, while s_g captures whether this signal is increasing, decreasing, or stable.

Each cluster C_k also has a dynamics center, computed from the micro-buckets currently assigned to it:

$$z_k = \frac{1}{|\mathcal{G}_k|} \sum_{G_g \in \mathcal{G}_k} z_g. \tag{4}$$

After standardizing the level and slope dimensions, a micro-bucket currently assigned to C_k is compared only with C_k and its adjacent clusters:

$$j^\star(g) = \arg \min_{j \in \{k-1,k,k+1\}} \|\tilde{z}_g - \tilde{z}_j\|_2^2. \tag{5}$$

If the same neighboring cluster is consistently closer across multiple rebucketing checks, the micro-bucket becomes a correction candidate.

To avoid one-way drift, the main correction operation is an adjacent mutual swap. If one micro-bucket in C_k is closer to C_{k+1} and another micro-bucket in C_{k+1} is closer to C_k, DURCL swaps them. This keeps cluster sizes stable and prevents the rebucketing module from replacing the scheduler. Therefore, the scheduler remains responsible for curriculum progression, while rebucketing only corrects local structural mismatches.

Algorithm 1 DURCL: Adaptive Cluster Scheduling with Micro-bucket Rebucketing

Require: Training set $\mathcal{D}$, initial clusters $\mathcal{C} = \{C_0, \ldots, C_{K-1}\}$, policy model π_θ

Split each initial task–cluster group into micro-buckets $\{G_g\}$
Initialize cluster values $\bar{A}_k \leftarrow 0$ and draw counts $n_k \leftarrow 0$
for training step $t = 1, \ldots, T$ **do**
  Compute cluster scores

$$U_k = \bar{A}_k + \beta\sqrt{\frac{2\log(N+1)}{n_k+1}}, \qquad p_k = \frac{\exp(U_k/\tau)}{\sum_j \exp(U_j/\tau)}$$

  Sample a cluster $C_k \sim p$ and draw a batch B_t from C_k
  Generate rollouts with π_θ, compute rewards, and estimate advantages $\{\hat{A}_i\}_{x_i \in B_t}$
  Update π_θ using the RL objective
  Update $\bar{A}_k$ and micro-bucket histories using $|\hat{A}_i|$
  if $t \geq T_{\text{warmup}}$ and t is a rebucketing checkpoint **then**
    For each eligible G_g, fit its recent dynamics signature

$$z_g = (\ell_g, s_g)$$

    For each cluster C_k, compute its current dynamics center

$$z_k = \frac{1}{|\mathcal{G}_k|} \sum_{G_g \in \mathcal{G}_k} z_g$$

    Standardize the level and slope dimensions of all z_g and z_k
    For each $G_g \in C_k$, find the closest current-or-adjacent cluster

$$j^\star(g) = \arg\min_{j \in \{k-1,k,k+1\}} \|\tilde{z}_g - \tilde{z}_j\|_2^2$$

    Mark G_g if the same neighboring $j^\star(g) \neq k$ is selected consistently across checks
    For each adjacent pair (C_k, C_{k+1}), swap the highest-margin mutual candidates:

$$G_g : C_k \to C_{k+1}, \qquad G_h : C_{k+1} \to C_k$$

  end if
end for
return Fine-tuned policy π_θ

5 Experiments

5.1 Step-200 Math+Zebra Validation

The clearest positive result appears in the Math+Zebra two-task comparison. At step 200, the scheduler without rebucketing improves over random sampling on both metrics.

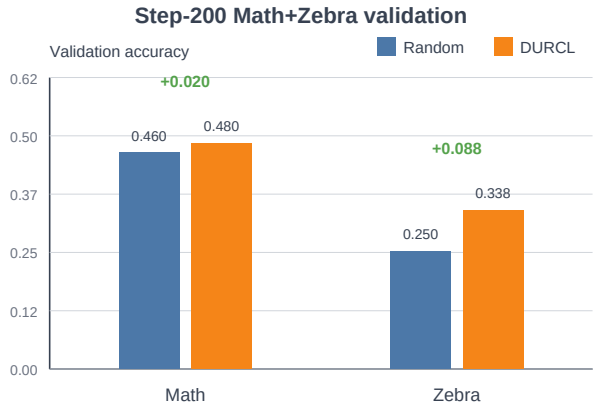

Figure 2: Step-200 validation on Math and Zebra. The no-rebucket DURCL sampler improves Math by +0.020 and Zebra by +0.088 over the random baseline.

Table 2: Math+Zebra step-200 validation.

Run	Step	Math	Zebra
Random baseline	200	0.460	0.250
DURCL sampler, no re-bucket	200	0.480	0.338

5.2 UCB Score Drift Diagnostic

The UCB trace gives a mechanism-level check that the scheduler is not behaving like a fixed sampler. Figure 3 plots the five cluster scores from step 60 to step 200. These values should be read as UCB scores, not normalized probabilities. The ranking changes substantially over training: C0 decreases from 0.329 at step 60 to 0.081 at step 200, while C3 rises from 0.238 to 0.295 and C4 remains high at 0.253. In other words, early sampling pressure is concentrated on C0/C1, but later pressure shifts toward C3/C4.

This supports the claim that UCB is doing useful adaptive work: the selected clusters are changing with the estimated learner signal and the exploration term, rather than staying near the initial difficulty prior. It also exposes a drift issue. If the scores move strongly toward high-index clusters, then downstream training can be affected by the resulting cluster and task-composition shift. This is why the rebucketing and composition diagnostics in Experiment 1 are necessary.

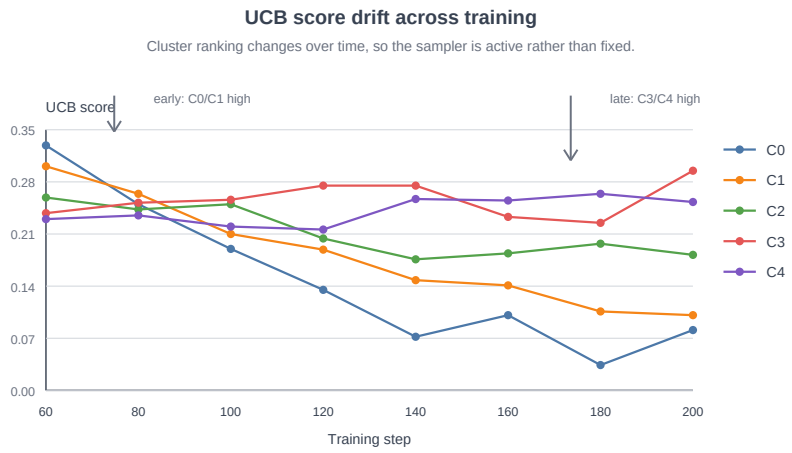

Figure 3: UCB score drift from step 60 to step 200. Low-index clusters C0/C1 have the largest scores early, while C3/C4 dominate later. This is evidence that the UCB sampler is responsive, and it motivates monitoring drift in cluster and task composition.

5.3 Experiment 1: Difficulty-Label Initialization and Composition Drift

Experiment 1 studies whether online rebucketing can correct imperfect initial curriculum buckets. The initial buckets are deliberately coarse: examples are initialized only from raw difficulty labels, without accuracy-based calibration. The initial accuracy test already shows why this is rough. Zebra accuracies are 0.30, 0.20, 0.25, and 0.15 for difficulties 1–4, so nominal difficulty is not strictly monotone with learner competence. Math accuracies are 0.70, 0.60, 0.35, 0.35, and 0.10 for difficulties 1–5, which is more monotone but lives on a much higher accuracy scale than Zebra. Figure 4 visualizes this mismatch.

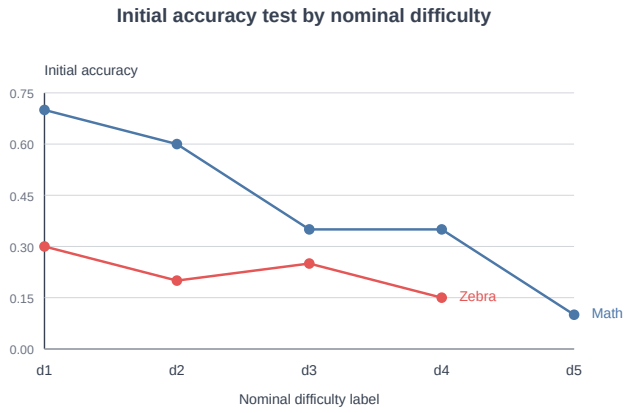

Figure 4: Initial accuracy test for label-only initialization. Zebra is non-monotone across nominal difficulty labels, while Math and Zebra have different accuracy scales. This motivates accuracy-based initialization as the next step.

Even under this rough initialization, rebucketing discovers meaningful redistribution patterns. The difficulty-init report gives a step-300 re-bucket result of 0.520 on Math and 0.287 on Zebra, compared with a baseline 200–300 reference of 0.560 on Math and 0.275 on Zebra. Thus rebucketing is not a simple final-accuracy win; its strongest current value is diagnosing and correcting imperfect buckets. Accuracy-based initialization is the next step to test whether better initial buckets lead to cleaner and more useful corrections.

The more useful evidence is the cluster-composition shift. Figure 5 reports both micro-bucket counts and sample counts. Initially, C0–C3 each contain 5 Math and 5 Zebra micro-buckets, while C4 is already Math-only because Zebra has only four difficulty clusters but Math has five. After rebucketing, C0 grows from 80 to 128 Math samples and 100 to 120 Zebra samples; C1 becomes Zebra-heavy, changing from 5 Math and 5 Zebra micro-buckets to 2 Math and 6 Zebra micro-buckets; C3 loses Zebra, dropping from 100 to 60 Zebra samples; and C4 remains Math-only while shrinking from 5 to 4 Math micro-buckets. This is meaningful structural correction, but it is also composition drift: DURCL is not merely correcting difficulty within a fixed task mixture.

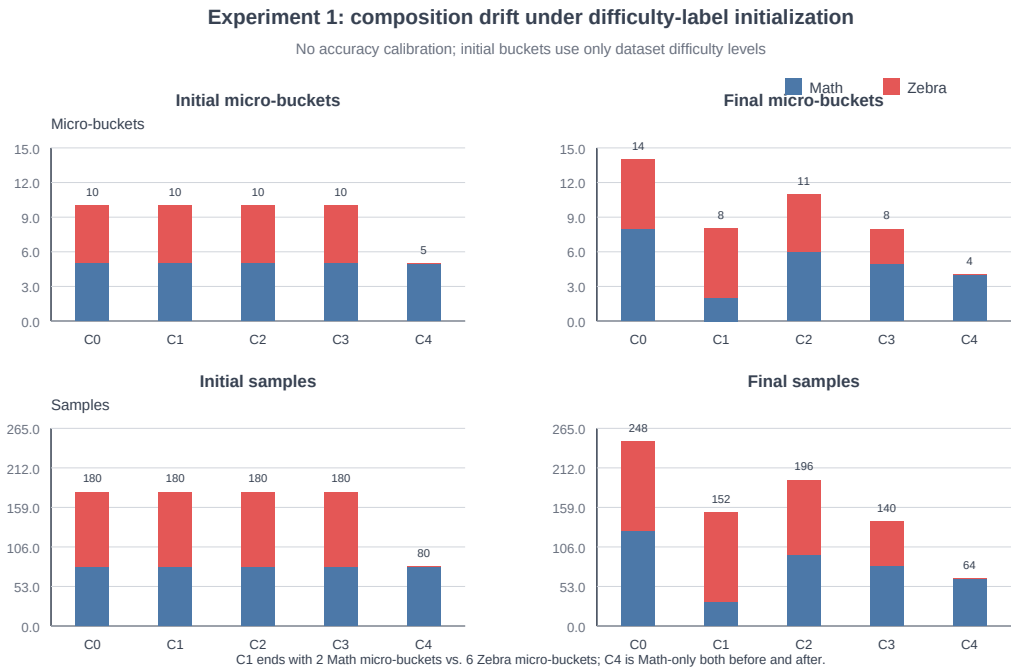

Figure 5: Experiment 1 composition drift under difficulty-label initialization. Rebucketing changes both micro-bucket composition and sample composition: C0 grows mainly through Math inflow, C1 becomes Zebra-heavy, C3 loses Zebra, and C4 remains Math-only.

The direction of the net changes is also meaningful. The checked-in result summary does not include raw migration-event logs, so Figure 6 visualizes a minimal adjacent-flow interpretation of the aggregate micro-bucket changes rather than raw per-step transitions. Under this interpretation, the off-diagonal mass is adjacent and moves toward lower-index buckets: for example, Math has a large C1 $\rightarrow$ C0 flow, and Zebra has C3 $\rightarrow$ C2 flow. This is consistent with a curriculum interpretation rather than random reassignment. In particular, Math C0 expands substantially, and that bucket corresponds to the highest initial Math accuracy group.

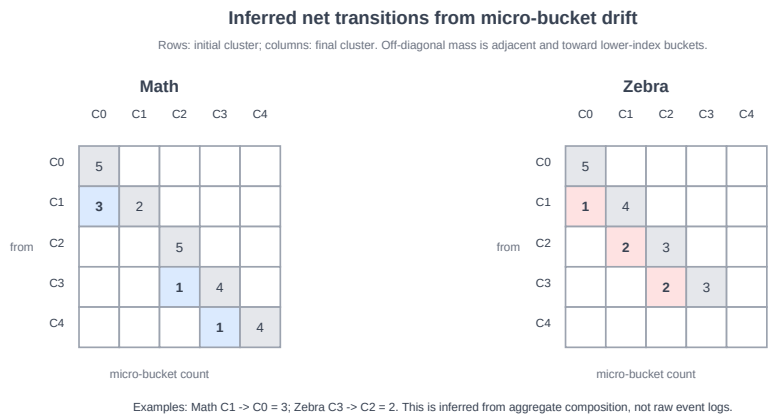

Figure 6: Inferred net transition matrix from initial/final micro-bucket counts. Rows are initial clusters and columns are final clusters. The non-diagonal changes are adjacent and toward lower-index buckets, supporting the claim that rebucketing is structured rather than random.

5.4 Experiment 2: Accuracy-Based Initialization

Experiment 2 repeats the 200-step comparison under accuracy-based initialization. This is the direct test of the next hypothesis from Experiment 1: if the initial buckets are built from learner accuracy rather than raw difficulty labels, then the scheduler should start from cleaner groups and rebucketing should become easier to interpret. The no-rebucket validation result is now available; the corresponding rebucketing run is still pending.

Table 3: Experiment 2 under accuracy-based initialization. The no-rebucket validation run is available; the rebucketing run is still pending.

Run	Step	Math	Zebra
Acc-init, no re-bucket	100	0.480	0.263
Acc-init, no re-bucket	200	0.500	0.300
Acc-init, re-bucket	200	pending	pending

The available no-rebucket result improves over the random step-200 reference of 0.460 on Math and 0.250 on Zebra. This is consistent with the interpretation that accuracy-based initialization is cleaner than raw difficulty labels. The missing rebucketing result remains the key test of whether micro-bucket correction adds value after a better initialization.

6 Discussion

The evidence supports a conservative interpretation. DURCL can produce useful adaptive sampling: in the two-task result, the no-rebucket sampler improves both Math and Zebra at step 200, and the accuracy-initialized no-rebucket run improves to 0.500 on Math and 0.300 on Zebra. DURCL also produces interpretable structural corrections: rebucketing moves micro-buckets in coherent ways under the difficulty-only initialization.

However, the current evidence is still limited. The rebucketing result is stronger as a structural-correction signal than as a final-accuracy claim, and the accuracy-initialized rebucketing run is still pending. The composition-drift analysis also shows that micro-bucket correction can change the Math/Zebra mixture inside clusters. This suggests that the current difficulty-only initialization is not well aligned with the learner's actual state and that task-composition drift must be monitored explicitly.

The most defensible conclusion is that DURCL demonstrates plausible adaptive environment selection behavior in the Math+Zebra setting, but the present implementation is still an early-stage mechanism rather than a robust training recipe.

7 Limitations and Future Work

The current evidence is limited by single-run comparisons, missing confidence intervals, and difficulty-only initialization. Runtime artifacts such as logs and checkpoints are intentionally git-ignored, so this paper relies on checked-in summaries and CSV files rather than full run reproduction.

The highest-priority next step is completing the accuracy-initialized rebucketing run. Initial buckets should be formed from observed model pass rate per task/difficulty level, not only nominal dataset difficulty. This should reduce noisy drift and make rebucketing corrections easier to interpret causally. Future experiments should also use multi-seed comparisons, report per-difficulty performance, add task-balance constraints or diagnostics, and separate within-task difficulty correction from cross-task composition shifts.

8 Conclusion

DURCL implements learner-state-aware environment selection for Math+Zebra RL-based LLM post-training. It combines UCB sampling, decayed advantage state, and conservative micro-bucket rebucketing. The current results show meaningful mechanism behavior and step-200 gains over random sampling, while also revealing composition drift. The project therefore establishes a useful direction and a working implementation, while identifying accuracy-initialized rebucketing and stricter evaluation as necessary next steps.

References

[1] Peter Auer, Nicolo Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3):235–256, 2002. doi: 10.1023/A:1013689704352. URL https://www.cs.utexas.edu/~shivaram/readings/b2hd-AuerCF2002.html.

[2] Yoshua Bengio, Jerome Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th International Conference on Machine Learning*, 2009. URL https://icml.cc/2009/papers/119.pdf.

[3] Qwen Team. Qwen2.5 technical report, 2024. URL https://arxiv.org/abs/2412.15115.

[4] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347.

[5] verl community. verl: Volcano engine reinforcement learning for llms. https://github.com/agentica-project/verl, 2024. URL https://github.com/agentica-project/verl.

[6] Zhenting Wang, Guofeng Cui, Kun Wan, and Wentian Zhao. Dump: Automated distribution-level curriculum learning for rl-based llm post-training, 2025. URL https://arxiv.org/abs/2504.09710.

Credit

This report is based on the group's DURCL implementation, checked-in experiment summaries, and repository documentation. OpenAI Codex was used as a support tool for organizing the report draft, generating plotting scripts, formatting LaTeX text, and checking consistency with the course guideline. Core project decisions, implementation artifacts, and experimental results come from the project repository. External papers, repositories, and tools are cited in the References section.